

Natural Language Processing

AIM-829

MODULE INSTRUCTOR: DR. TULIKA SAHA

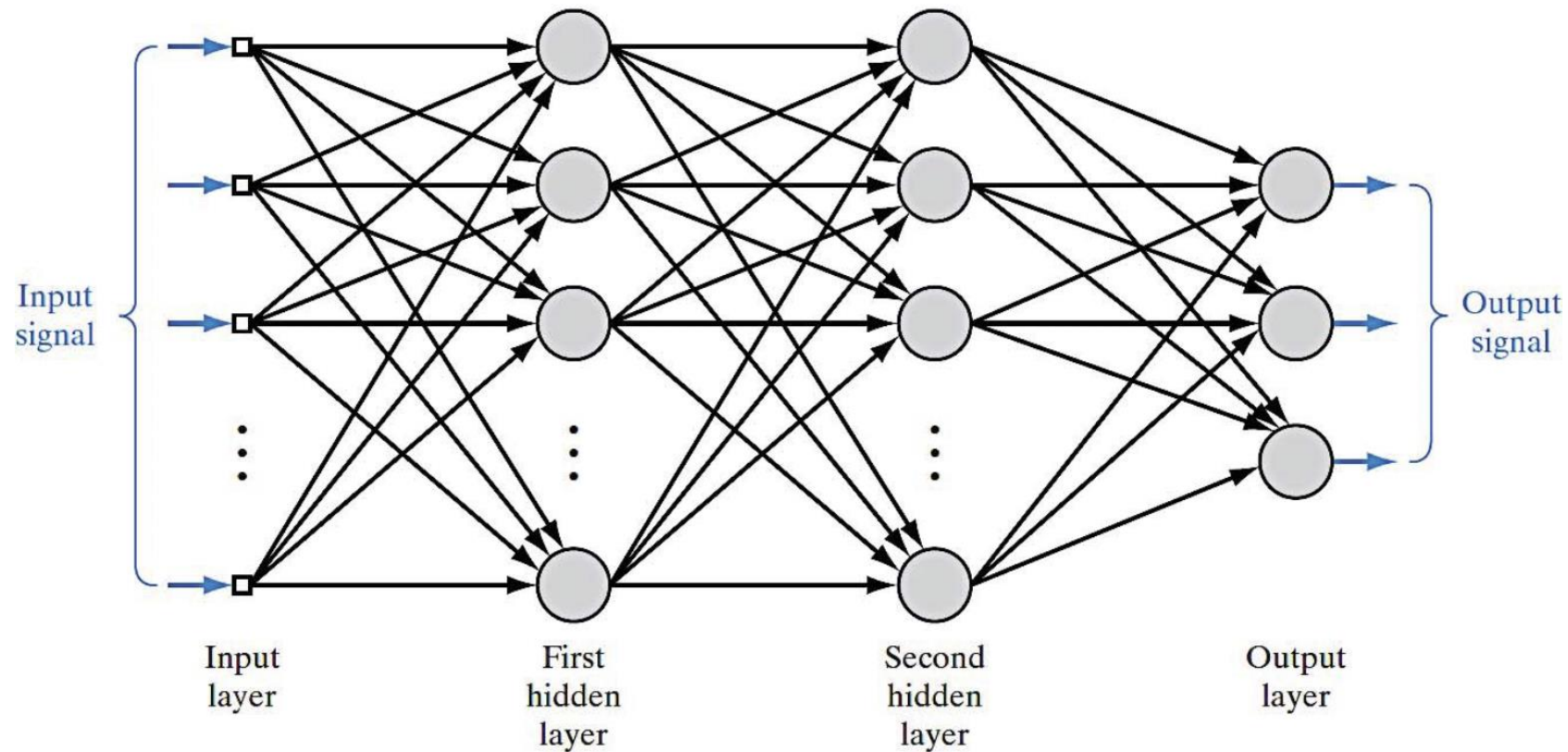
EMAIL: TULIKA.SAHA@IIITB.AC.IN

OFFICE: 102-E (ARYABHATA)

Slides are adopted from the Stanford course 'NLP with DL' by C. Manning, UMass course 'Advanced NLP' by M Iyer, 'NLP with DL' by P. Bhattacharyya and LCS at IIT-Delhi

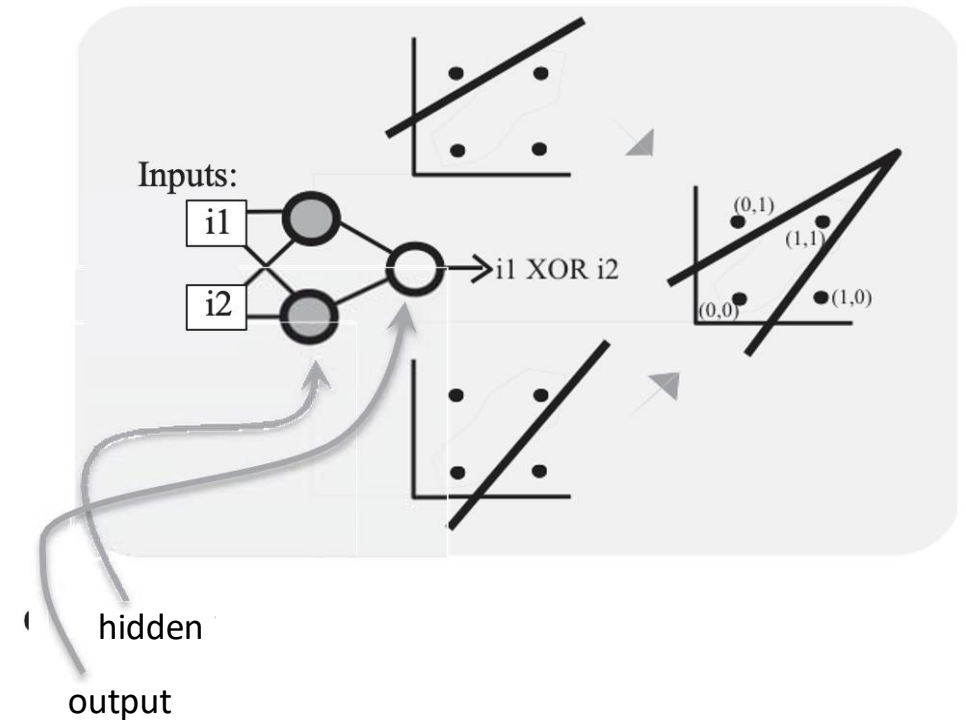
Mandate 2 - Neural NLP

Multi-layer Perceptrons (MLPs)



The function of hidden layers

- The presence of hidden layers can allow complex representations. Consider the XOR problem for instance, which is not a linearly separable one:
 - Each of the **hidden nodes** above, **finds a partial** solution to the problem
 - Then, the **output** node **combines the two partial** solutions and makes the problem separable.



The Back-Propagation (BP) Algorithm

□ This is a very popular computationally efficient (but not unique) method for training MLPs. Although it is not optimal it discards the pessimism of some SLP pioneers about the uselessness of multi-layer learning!

□ The algorithm is as follows:

- We define the error signals at the j^{th} output neuron level with target output d_j and at the network level as:

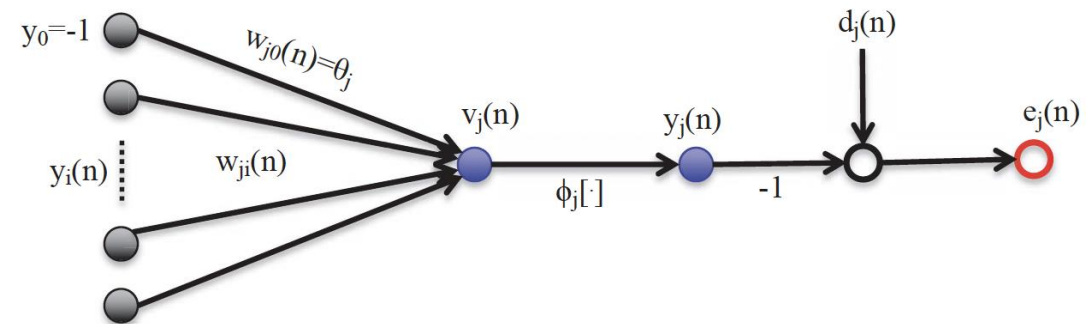
Output neuron error: $e_j(n) = d_j(n) - y_j(n)$

Output network error: $E(n) = \frac{1}{2} \sum_{j \in \text{Output}} e_j^2(n)$

Average net. output error: $E_{\text{avg}} = \frac{1}{N} \sum_{n=1}^N E(n)$ (for N data patterns)

where, Induced local field: $v_j(n) = \sum_{i=0}^p w_{ji}(n) y_i(n)$

Neuron output: $y_j(n) = \phi_j[v_j(n)]$



The Back-Propagation (BP) Algorithm

- We now wish to train the MLP using our available data patterns $(\mathbf{x}_i, \mathbf{d}_i)$, $i=1, \dots, N$.
 - But the goal is always the same:
 - Adapt each synaptic weight: $\Delta w_{ji}(n) \equiv w_{ji}(n+1) - w_{ji}(n)$
 - First we calculate the derivative of the network error $E(n)$ at time n with respect to the synaptic weight that needs to be adjusted to minimise this error:

$$\begin{aligned} \frac{\partial E(n)}{\partial w_{ji}(n)} &= \underbrace{\frac{\partial E(n)}{\partial e_j(n)}}_{e_j(n)} \underbrace{\frac{\partial e_j(n)}{\partial y_j(n)}}_{-1} \underbrace{\frac{\partial y_j(n)}{\partial v_j(n)}}_{\phi'_j[v_j(n)]} \underbrace{\frac{\partial v_j(n)}{\partial w_{ji}(n)}}_{y_i(n)} = -\overbrace{e_j(n) \phi'_j[v_j(n)]}^{\delta_j(n)} y_i(n) = \\ &= -\delta_j(n) y_i(n) \end{aligned}$$

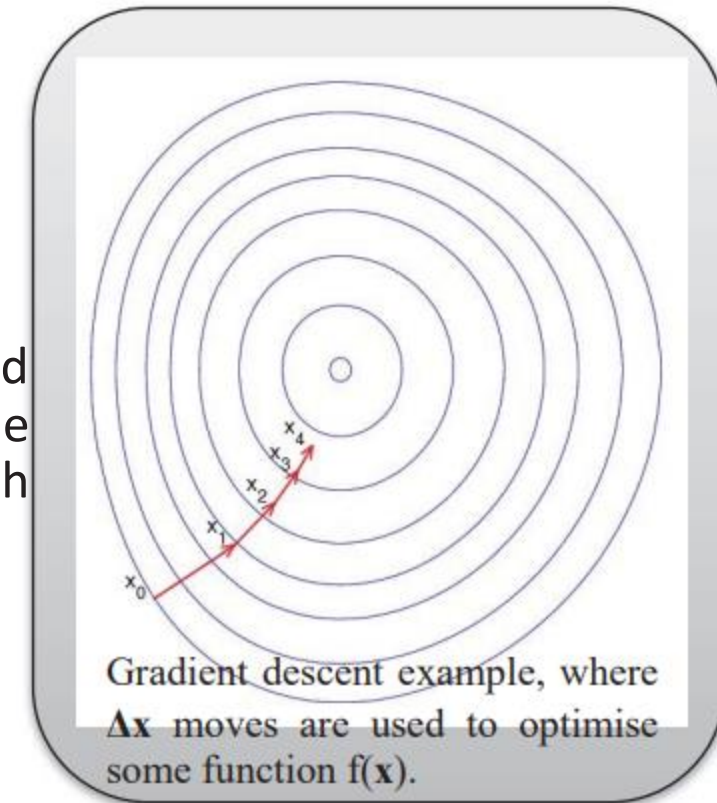
The Back-Propagation (BP) Algorithm

- So, given a learning rate η , a simple gradient descent optimisation, would move in the weight space towards the direction of:

$$\Delta w_{ji}(n) = -\eta \frac{\partial E(n)}{\partial w_{ji}(n)} = \eta \delta_j(n) y_i(n)$$

- This direction which minimises $E(n)$, is based on the newly introduced local gradient of the j^{th} neuron, which is defined as the product of the error signal of that neuron and the derivative of its activation with respect to the ILF of the neuron:

$$\delta_j(n) \equiv e_j(n) \phi'_j[v_j(n)]$$



The Back-Propagation (BP) Algorithm

- Thus, to calculate Δw_{ji} the error signal at the output of the j^{th} neuron is needed. But **not all** neurons have directly known output! This gives rise to two cases:
 - **j is an output neuron:** Here it is easy to calculate the weight change Δw_{ji} , since $e_j(n) = d_j(n) - y_j(n)$, and d_j is known by the training dataset.
 - **j is a hidden neuron:** Here the error is **not** directly known, **but** these neurons surely have a responsibility for the error observed in the output layer.
- But how can we penalise or reward the hidden neuron when their errors $e_j(n)$ are not directly available because their desired responses $d_j(n)$ are not? This is an instance of the **Credit Assignment Problem!**
- This is solved by "back-propagating" the signal error from the output layer through the previous hidden layers, as follows:
 - Using the previous definitions, we can **conceptually redefine** the local gradient as:

$$\delta_j(n) = -\frac{\partial E(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} = \boxed{-\frac{\partial E(n)}{\partial y_j(n)} \phi'_j[v_j(n)]}^2 = e_j(n) \phi'_j[v_j(n)]$$

The Back-Propagation (BP) Algorithm

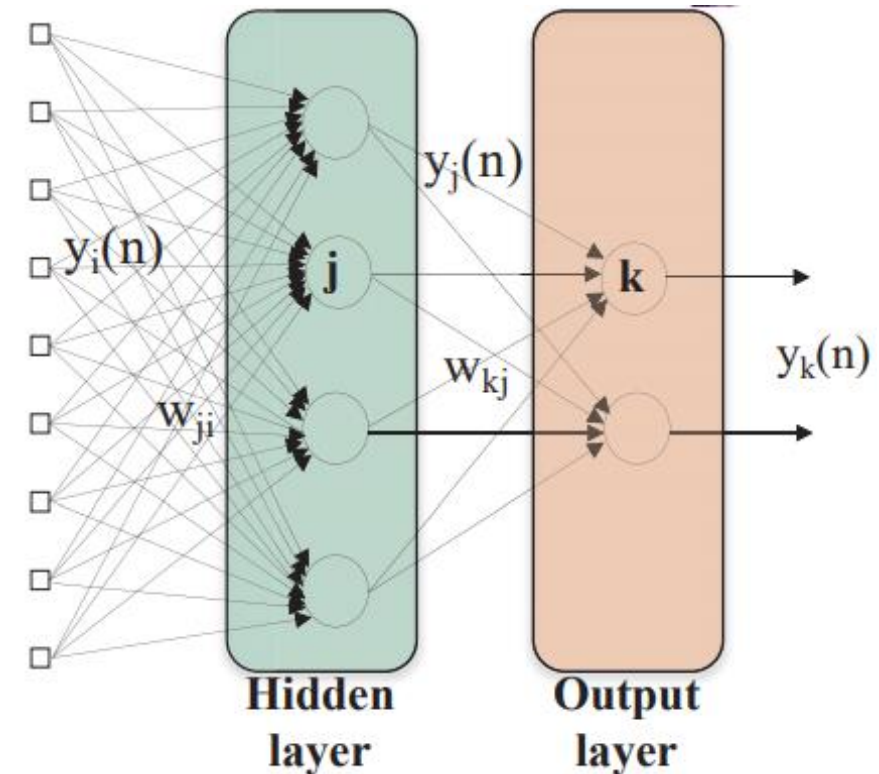
$$\frac{\partial E(n)}{\partial y_j(n)} = - \sum_k e_k(n) \underbrace{\phi'_k[v_k(n)]}_{\delta_k(n)} w_{kj}(n)$$

- Finally, inserting the above into Eq(2), we get the back-propagation formula for the local gradient in the j^{th} hidden neuron:

$$\delta_j(n) = \phi'_j[v_j(n)] \sum_k \delta_k(n) w_{kj}(n)$$

B

- Eq(B) allows us to apply the weight change delta-rule or Eq(1) to hidden neurons. It can be seen that the δ_j of a hidden node can be calculated in terms of the δ_k of the k^{th} neurons connected (i.e., receiving signals) from neuron j.
- This means that Eq(B) is valid for j being a neuron in any hidden layer, whilst neuron k can be a neuron from the next hidden or the output layer.
- Overall, Eq(A) is used for the output layer, and Eq(B) for all the hidden ones.



The Back-Propagation (BP) Algorithm: Full Summary

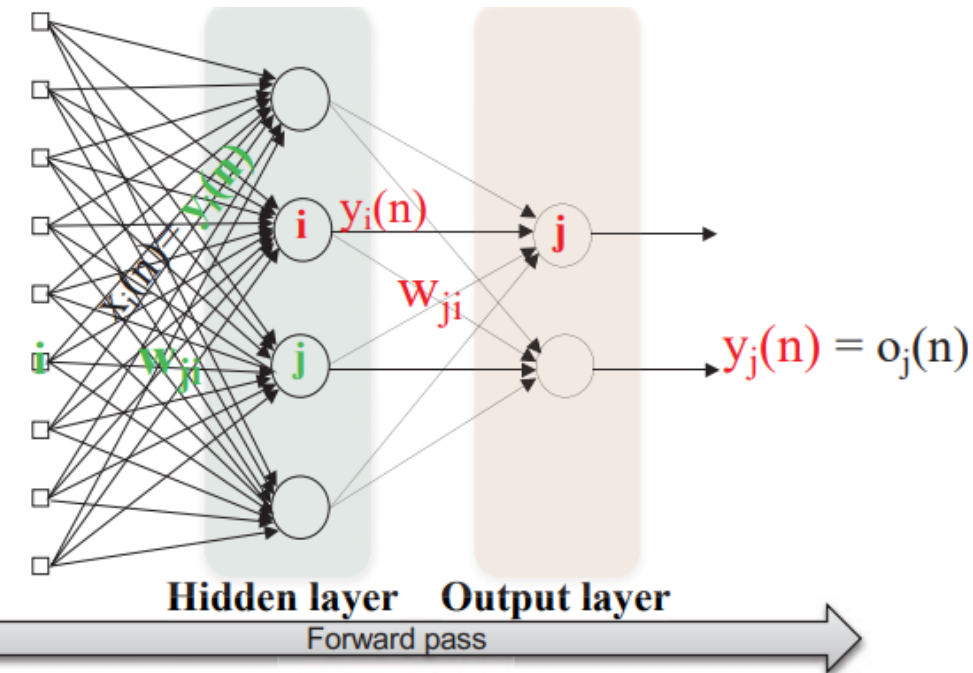
- The BP algorithm for training MLPs consists of two passes of computation, which are alternatively applied.

Forward Pass

- All weights remain fixed.
- We start from the first hidden layer and for all its neurons we calculate their outputs and ILFs as:

$$y_j(n) = \phi_j[v_j(n)], \quad v_j(n) = \sum_{i=0}^p w_{ij}(n)y_i(n)$$

- If the above is applied in the first hidden layer all $y_i(n)$ simply represent the network input signal $X_i(n)$
- We repeatedly move to the next layer, until we reach the output layer where $y_j(n)$ is the j^{th} network output, typically also written as $o_j(n)$.
- If the above is applied to the 2nd, 3rd, etc. Hidden layer or the output layer, $y_i(n)$ is output signal of the i^{th} node in the previous hidden layer.



The Back-Propagation (BP) Algorithm: Full Summary

Backward Pass

- This pass starts from the last (the output layer) and works backwards in a layer-by-layer fashion, until it reaches the first hidden layer
- All input signals to nodes are clamped during this pass
- For each j^{th} node of the current layer, it calculates the local gradient using:

$$\odot \text{ Eq(A), i.e. } \delta_j(n) = e_j(n) \phi'_j[v_j(n)]$$

if the node is in the output layer, or

$$\odot \text{ Eq(B), i.e. } \delta_j(n) = \phi'_j[v_j(n)] \sum_k \delta_k(n) w_{kj}(n)$$

if the node is in any hidden layer.

- All the local gradients are then used in Eq(1) to calculate and adapt the synaptic weights so that to reduce the network error:

$$\Delta w_{ji}(n) = -\eta \frac{\partial E(n)}{\partial w_{ji}(n)} = \eta \delta_j(n) y_i(n)$$

